Rapport final technique

Réseau virtuel de machines en Bash

LO14 - UTT - 2026-2027

Jules et Selyan - 8 novembre 2026

Version de travail technique à relire avant remise. Code de référence : `9a56994`. La répétition humaine sur le poste du binôme reste à effectuer.

1. Objectif et fonctionnalités

`rvsh` fournit un shell Bash pour un réseau virtuel décrit par des fichiers locaux. Les utilisateurs se connectent à des machines simulées, sous réserve d'une autorisation et d'une authentification. L'administrateur gère les machines, les comptes et leurs accès depuis `hostroot`.

Domaine	Commandes réalisées
Lancement	`rvsh -admin`, `rvsh -connect machine utilisateur`.
Consultation	`who`, `rusers`, `rhost`.
Navigation et identité	`rconnect`, `exit`, `su`, `passwd`.
Informations	`finger`, `afinger`.
Administration	`host`, `user`, attribution et retrait des droits.
Messagerie	`write`, `wall`, `wall -n`.

Le travail porte sur une simulation locale partagée entre terminaux. Il n'exige ni protocole TCP ni création de véritables machines virtuelles. Le précédent serveur d'archives en C est conservé dans la branche `archive-projet-2025`, indépendamment de la version Bash.

2. Organisation du code

Le lanceur `rvsh` vérifie les arguments, charge les modules et authentifie le premier utilisateur. `storage.sh` concentre les primitives liées aux données et au verrouillage. `session.sh` organise les connexions et les notifications. `commands.sh` traduit les commandes saisies en opérations sur ce modèle.

Le traitement repose sur un aiguillage fermé : seul un ensemble connu de commandes est accepté. Le programme ne transmet pas la ligne saisie à `eval` ou à un autre interpréteur système. Les commandes qui transportent un texte libre séparent la destination du reste du message sans tenter d'évaluer les caractères spéciaux.

3. Modèle des données et concurrence

Le dossier du réseau contient `hosts/`, `users/`, `sessions/`, `inbox/` et `mail/`. Chaque compte possède un hachage de mot de passe, une liste de machines autorisées et une fiche d'informations. Les connexions multiples du même utilisateur restent distinctes grâce à des identifiants de session propres à chaque connexion.

Un verrou `flock` protège les accès cohérents aux fichiers partagés. Il évite notamment que deux administrateurs réécrivent simultanément le même fichier de droits. Le verrou est relâché après chaque opération et n'est pas conservé pendant la saisie d'un mot de passe.

La modification d'un mot de passe suit trois étapes : lecture d'une référence cohérente, saisie et calcul du nouveau hachage, puis contrôle de cette référence sous verrou avant remplacement. Si un autre terminal a modifié le compte entre-temps, l'opération est refusée et doit être recommencée.

La persistance est assurée par les fichiers locaux. Le test de fermeture puis réouverture vérifie la conservation des machines, droits, informations complémentaires et messages différés. Le projet ne fournit pas de transaction générale avec journal de reprise après panne disque ; cette limite dépasse le modèle pédagogique retenu.

4. Authentification et gestion des droits

Le premier mode admin initialise root et `hostroot` après choix du mot de passe. Les mots de passe sont saisis sans affichage et stockés sous forme SHA-512 crypt avec un sel aléatoire fourni par OpenSSL. Les fichiers créés sont privés pour le compte Linux qui exécute le programme.

`user add` crée un compte sans autorisation de connexion. `user grant` et `user revoke` modifient les droits. `rvsh -connect` et `rconnect` contrôlent l'accès à la machine cible. `su` demande le mot de passe du nouveau compte et vérifie son accès à la machine courante.

Les commandes administrateur exigent root dans un contexte ouvert par `-admin`. Un utilisateur ordinaire doit passer par ce mode pour administrer le réseau. La suppression d'une machine ou d'un compte utilisé, ainsi que le retrait d'un droit utilisé, sont refusés. Ces règles évitent de laisser une session active référencer une ressource supprimée.

5. Connexions imbriquées et cycle de vie

Une pile conserve les identités et machines des connexions. Pour A, puis B, puis C, le premier `exit` ferme C, le deuxième ferme B et le troisième ferme A. Les entrées antérieures restent recensées comme connectées. `su` remplace l'identité du sommet, sans changer la machine ni ajouter une nouvelle machine à la pile.

Le PID seul ne suffit pas à identifier durablement le propriétaire d'une session : Linux peut réutiliser un numéro. La version validée ajoute un identifiant de démarrage du système et le temps de démarrage du processus, lus dans `/proc`. Une session dont ces métadonnées ne correspondent plus est retirée, même si le PID existe toujours. Les processus zombies ne sont pas considérés comme propriétaires valides.

La fermeture normale, EOF et les signaux INT, HUP et TERM déclenchent le nettoyage. SIGKILL ne permet aucun traitement par le processus tué ; les opérations ultérieures détectent les entrées mortes et les suppriment. Les anciennes entrées dépourvues d'identité complète sont aussi retirées. Pour une mise à jour du programme, les anciennes sessions doivent être fermées avant de relancer la nouvelle version.

6. Messagerie

`write utilisateur@machine message` livre une copie à chaque connexion correspondant à la destination. Une destination absente provoque un refus. Le message indique l'identité virtuelle de l'expéditeur.

`wall` diffuse une copie à chaque session connectée. Avec `-n`, les comptes entièrement hors ligne reçoivent également un message persistant. À la première connexion suivante du compte, ce message passe dans la boîte de la session et la copie différée est consommée.

Le processus de notification lit les boîtes sous verrou puis les vide. Son descripteur de verrou est rouvert pour ne pas partager l'état de verrouillage du parent. Il utilise une tentative non bloquante pour pouvoir s'arrêter lors du nettoyage. La consultation est espacée d'environ 0,2 seconde. Une notification peut couper visuellement le prompt en cours de saisie.

7. Validation et correction d'un défaut

La recette automatisée utilise de véritables processus Bash reliés à des pseudo-terminaux. Elle s'exécute sous Linux, dans un conteneur sans réseau et avec le code monté en lecture seule. Les comptes et répertoires de test sont jetables.

Contrôle exécuté	Résultat
Vérification de syntaxe Bash	Valide.
Analyse ShellCheck avec modules sourcés	Aucun diagnostic.
Interface de lancement	11 vérifications réussies.
Intégration Linux	10 scénarios réussis en 7,569 secondes.

Les scénarios couvrent l'authentification, les droits, les connexions imbriquées, `su`, les changements de mots de passe, la persistance, les connexions multiples, les messages immédiats et différés, les accès concurrents et les fermetures par signaux.

Le défaut de session résiduelle a été reproduit avant correction : une entrée ancienne portant un PID encore vivant restait dans `rusers`, qui affichait deux sessions root au lieu d'une. Après ajout de l'identité du processus, le même cas retire l'entrée et sa boîte de messages périmés tout en conservant la session légitime. Le test fait partie de la suite finale.

Une recette interactive supplémentaire a été exécutée dans deux terminaux séparés : authentification, droits, parcours A/B/C, retours, message immédiat, reconnexion avec message différé et absence de session résiduelle après fermeture. Son compte rendu figure dans `docs/RECETTE.md`. Elle complète les tests sans remplacer la répétition personnelle du binôme.

8. Limites et conclusion technique

Le projet respecte le périmètre fonctionnel décrit par le sujet et dispose d'une validation automatisée sous Linux. Il nécessite un stockage local partagé et `/proc`. Il n'est pas conçu pour isoler des utilisateurs Linux différents ni pour servir de système distribué entre plusieurs ordinateurs.

Les livraisons de messages s'appuient sur des fichiers et un vidage des boîtes ; elles ne fournissent pas d'accusé de réception persistant en cas de panne pendant l'affichage. L'interface peut être améliorée pour redessiner le prompt lors d'une notification. La reprise après incident disque pourrait aussi être renforcée.

Avant remise, le binôme doit exécuter la démonstration manuelle, confirmer ses choix de comportement et être capable d'expliquer les modules, le verrou et les tests. Ce rapport ne prétend pas que cette répétition humaine a eu lieu. La liste exacte des commandes de validation figure dans `docs/VALIDATION.md` ; le sujet officiel est conservé dans le dépôt.